ADA2026

Assignment 3

Trees, Hashing and Complexity Analysis

Aye Khin Khin Hpone (Yolanda Lim)-st125970

Question 1: Binary Search Trees

1.1 Building the BST

To insert keys into an empty BST, each new value is compared against the current node and directed left if smaller or right if larger, repeating until an empty slot is found. The step-by-step insertion trace is as follows:

1. Insert 40 → becomes the root (tree is empty).
2. Insert 20 → 20 < 40, go left → left child of 40.
3. Insert 60 → 60 > 40, go right → right child of 40.
4. Insert 10 → 10 < 40 (left), 10 < 20 (left) → left child of 20.
5. Insert 30 → 30 < 40 (left), 30 > 20 (right) → right child of 20.
6. Insert 50 → 50 > 40 (right), 50 < 60 (left) → left child of 60.
7. Insert 70 → 70 > 40 (right), 70 > 60 (right) → right child of 60.

BST after inserting: 40, 20, 60, 10, 30, 50, 70

```
          40          ← root
         /   \
        20    60
       / \   / \
      10  30 50  70

Depth 0:      40
Depth 1:    20   60
Depth 2:   10 30 50 70
```

Figure 1 — Balanced BST (height = 2, optimal for 7 nodes)

1.2 In-Order Traversal

In-order traversal visits nodes in the pattern Left → Root → Right, which always produces a sorted sequence from a valid BST:

In-Order Result

10 → 20 → 30 → 40 → 50 → 60 → 70 This is the ascending sorted sequence of all seven keys, confirming that the BST property is correctly satisfied throughout the tree.

1.3 Time Complexity of BST Operations

BST performance is entirely tied to tree height, since every operation traces a root-to-leaf path. There is no self-balancing mechanism, so the height depends completely on insertion order:

Operation	Best Case	Average Case	Worst Case
Search	O(1)	O(log n)	O(n)
Insert	O(1)	O(log n)	O(n)
Delete	O(1)	O(log n)	O(n)

Best case O(1) occurs when the target is the root itself. Average case O(log n) assumes randomly ordered input producing a balanced tree of height ≈ $\log_2 n$. Worst case O(n) occurs when the tree degenerates into a linear chain.

1.4 Degenerate BST — Worst-Case Behaviour

A degenerate BST occurs when keys are inserted in sorted or reverse-sorted order. Every new key becomes the rightmost (or leftmost) child of the previously inserted node, producing a linear chain with no branching — structurally identical to a linked list.

Example input sequence: 1, 2, 3, 4, 5, 6, 7

Degenerate BST from sorted input [1, 2, 3, 4, 5, 6, 7]

```
1  ← root
 \
  2
   \
    3
     \
      4
       \
        5
         \
          6
           \
            7  ← height = n = 7  (worst case)

Every node has only a right child.
Tree height = n  instead of the ideal log₂(7) ≈ 3.
```

Figure 2 — Degenerate BST: height = n, equivalent to a linked list

This is worst-case because every search, insertion, or deletion must traverse the full chain of n nodes before reaching its target. The height equals n rather than $\log_2 n$, causing all operations to degrade from O(log n) to O(n) — equivalent to a linked list.

Question 2: Red-Black Trees

2.1 The Five Properties of a Red-Black Tree

A Red-Black Tree is a BST that assigns each node a colour — red or black — and enforces five invariants that collectively guarantee the tree remains balanced:

- Property 1: Every node is coloured either RED or BLACK.
- Property 2: The root is always BLACK.
- Property 3: Every leaf (NIL sentinel node) is BLACK.
- Property 4: If a node is RED, both its children must be BLACK — no two consecutive red nodes on any path.
- Property 5: For every node, all simple paths from that node down to any descendant NIL leaf contain the same number of BLACK nodes (uniform black-height).

2.2 Why Red-Black Trees Guarantee O(log n) Search

The five properties collectively enforce a tight bound on tree height:

- Property 4 prevents two consecutive red nodes, so on any root-to-leaf path at most half the nodes can be red.
- Property 5 ensures every root-to-leaf path shares the same black-height bh.
- Combining both: the longest path ≤ 2 × shortest path, because it can alternate red-black nodes.
- A subtree with black-height bh has at least 2^bh − 1 internal nodes (proven rigorously in Question 6).
- Therefore bh ≤ $\log_2(n+1)$, and tree height h ≤ $2 \cdot \log_2(n+1)$ = O(log n).
- Search visits at most one node per level, so it takes at most h steps → O(log n) guaranteed.

2.3 BST Height vs Red-Black Tree Height

Property	Binary Search Tree (BST)	Red-Black Tree (RBT)
Height (best)	O(log n) — balanced	O(log n) — guaranteed
Height (worst)	O(n) — degenerate chain	≤ $2 \cdot \log_2(n+1)$ — bounded
Balancing	None — insertion-order dependent	Automatic via rotations + recolouring
Search guarantee	O(log n) average only	O(log n) always
Insertion overhead	O(log n) average	O(log n) with fix-up cost

Why RBTs remain balanced: every insertion or deletion that would violate one of the five properties immediately triggers recolouring and/or rotations (left or right). These fix-up operations restore all properties in O(log n) time while keeping the black-height uniform across every root-to-leaf path, preventing the degeneration that ordinary BSTs are susceptible to.

Question 3: Red-Black Tree Insertion

Colour convention: B = Black, R = Red. New nodes are always inserted as RED. Fix-ups (recolour / rotate) then restore all five RBT properties.

Step 1: Insert 10	
Before fix-up: (empty tree)	**After fix-up:** 10(B) ← root
Fix-up: Inserted 10 as RED. Root must be BLACK (Property 2) → recolour to BLACK. No rotations needed.	

Step 2: Insert 20

Before fix-up:

```
10(B)
```

After fix-up:

```
10(B)
  \
  20(R)
```

Fix-up: 20 > 10 → right child. Parent 10 is BLACK → no violation of Property 4. Tree is valid, no fix-up needed.

Step 3: Insert 30 — Left Rotation

Before fix-up:

```
10(B)
  \
  20(R)
    \
    30(R) ← VIOLATION: two consecutive reds
```

After fix-up:

```
    20(B)
   /   \
10(R)  30(R)
```

Fix-up: Violation: 20(R)→30(R) — consecutive reds (Property 4). Uncle of 30 is NIL (BLACK). Case 3 (right-right line): left-rotate on node 10, then recolour 20→BLACK, 10→RED.

Step 4: Insert 15 — Recolouring

Before fix-up:

```
    20(B)
   /   \
10(R)  30(R)
   \
   15(R) ← VIOLATION: parent 10 is red
```

After fix-up:

```
    20(B)
   /   \
10(B)  30(B)
   \
   15(R) ✓ (parent now BLACK)
```

Fix-up: 15 inserts as right child of 10. Parent 10 is RED and uncle 30 is also RED → Case 1 (uncle red): recolour 10→BLACK, 30→BLACK. Root 20 stays BLACK. No rotation needed.

Step 5: Insert 40

Before fix-up:

```
    20(B)
   /   \
10(B)  30(B)
   \     \
   15(R)  40(R)
```

After fix-up:

(no change)

Fix-up: 40 > 30 → right child of 30. Parent 30 is BLACK → no violation. Tree is already valid.

Step 6: Insert 5 — Final insertion

Before fix-up:

```
    20(B)
   /   \
10(B)  30(B)
 /  \    \
5(R) 15(R)  40(R)
```

After fix-up:

(no change — valid)

Fix-up: 5 < 20 (go left), 5 < 10 (go left) → left child of 10. Parent 10 is BLACK → no violation. No fix-up needed.

Final RBT after inserting: 10, 20, 30, 15, 40, 5

```
        20(B)        ← root
       /   \
    10(B)   30(B)
   /   \      \
 5(R)  15(R)   40(R)

Black-height of root = 2  (counting only black nodes, root exclusive)
Every root-to-NIL path: 20(B) → {10(B) or 30(B)} → NIL  = exactly 2 black nodes  ✓
```

Figure 3 — Final Red-Black Tree, all five properties satisfied

Verification of all five RBT properties:

- Property 1: All nodes are RED or BLACK ✓
- Property 2: Root 20 is BLACK ✓
- Property 3: All NIL sentinel leaves are BLACK ✓
- Property 4: No two consecutive reds — 5(R)'s parent is 10(B), 15(R)'s parent is 10(B), 40(R)'s parent is 30(B) ✓
- Property 5: Black-height = 2 on every root-to-NIL path ✓

Question 4: Hash Tables and Load Factor

Hash function: h(k) = k mod 10. Table size m = 10. Keys to insert: 15, 25, 35, 44.

4.1 Hash Index Computation

Key (k)	Calculation	Hash Index h(k)
15	15 mod 10	5
25	25 mod 10	5
35	35 mod 10	5
44	44 mod 10	4

Keys 15, 25, and 35 all map to index 5 — a collision cluster arising from the hash function itself, independent of the overall load factor.

4.2 Final Hash Table — Separate Chaining

Index	Chain (Separate Chaining)
0	(empty)
1	(empty)
2	(empty)
3	(empty)
4	[44] ← h(44) = 44 mod 10 = 4
5	[15] → [25] → [35] ← h(15)=h(25)=h(35)=5 (collision!)
6	(empty)
7	(empty)
8	(empty)
9	(empty)

Separate chaining resolves collisions by storing all keys that hash to the same index in a linked list. Bucket 5 holds a chain of three nodes: 15 → 25 → 35.

4.3 Load Factor

Load Factor $\lambda = n / m = 4 / 10 = 0.4$
$n = 4$ keys inserted, $m = 10$ buckets total. $\lambda = 0.4$ means the table is 40% occupied — well within the efficient range. Note that despite the low overall load factor, bucket 5 already holds a chain of three due to hash function clustering.

4.4 How Load Factor Affects Performance

Load Factor λ	Bucket State	Average Search Time	Assessment
$\lambda < 0.5$	Few collisions, mostly empty or singleton chains	≈ O(1)	Excellent
$0.5 \leq \lambda \leq 0.75$	Moderate collisions	O(1) to O(2)	Good — standard threshold
$\lambda > 1.0$	Many collisions, long chains	O(λ) per lookup	Degraded — resize needed
$\lambda = 0.4$ (ours)	Mostly empty — but bucket 5 has 3 keys	≈ O(1) average	Good — local clustering only

An important subtlety: even with a low overall load factor, a poor hash function can create clusters as seen in bucket 5. This demonstrates that λ measures average density, not worst-case chain length.

4.5 Search Complexity at High Load Factor

With separate chaining, search time breaks down as follows:

- Computing the hash index always costs O(1).
- Scanning the chain at that bucket costs O(chain length).
- Average chain length = λ, so average search = O(1 + λ).
- Worst case: all n keys collide into one bucket → chain length = n → search = O(n).

This worst case is the adversarial scenario explored in Question 7: inserting multiples of the table size forces all keys to the same bucket.

4.6 Strategies to Maintain Efficient Performance

- **Dynamic Resizing (Rehashing):** when λ exceeds a threshold (typically 0.75), allocate a new table roughly double the size and rehash all existing keys. This keeps λ below the threshold and amortises insertion cost to O(1) on average.
- **Better Hash Function / Universal Hashing:** functions such as MurmurHash or SipHash distribute keys uniformly across all buckets, minimising clustering. Universal hash families guarantee low expected collision probability against any fixed input set.

Question 5: Designing a Median Data Structure

5.1 Proposed Data Structure: Dual-Heap

The proposed solution maintains two heaps in parallel:

- **Lower max-heap L:** stores the smaller half of the dataset. Its root is the largest value in the lower half.
- **Upper min-heap U:** stores the larger half of the dataset. Its root is the smallest value in the upper half.

Invariant: |L| = |U| or |L| = |U| + 1
L is kept either equal in size to U, or exactly one element larger. The median is always accessible at the root of L (or the average of both roots when sizes are equal).

Dual-heap structure — example state with dataset {5, 8, 10, 20, 25, 30}

```
Lower Max-Heap (L)        Upper Min-Heap (U)
    [10]                      [20]
   /   \                     /   \
  [5]   [8]                 [30]   [25]

max(L) = 10 ≤ min(U) = 20 ← heap-order invariant holds
|L| = 3, |U| = 3 → equal sizes → Median = (10 + 20) / 2 = 15
```

Figure 4 — Dual-heap maintaining the lower and upper halves of the dataset

5.2 How Each Operation Works

Insert(x)

- If x ≤ max(L) (or L is empty), push x onto L; otherwise push x onto U.
- Rebalance: if |L| > |U| + 1, move max(L) to U; if |U| > |L|, move min(U) to L.
- Both heap operations cost O(log n) → Insert is O(log n).

Delete(x)

- To achieve O(log n), a position map (hash map of value → heap index) is required. Without it, locating x requires an O(n) linear scan, which would violate the O(log n) requirement.
- With the position map, lookup is O(1). Then swap x with the last element, reduce the heap size, and sift the replacement up or down → O(log n).
- Rebalance the two heaps using the same rule as Insert → O(log n) total.

FindMedian()

- If |L| > |U|: median = max(L) = L[0] — O(1) root access.
- If |L| = |U|: median = (max(L) + min(U)) / 2 — O(1) root access on both.

5.3 Pseudocode for FindMedian()

```
function FindMedian():
   if size(L) > size(U):
      return peek(L)              // max of lower heap — O(1)
   else:
      return (peek(L) + peek(U)) / 2  // average of both roots — O(1)

function Insert(x):
   if isEmpty(L) or x <= peek(L):
      push(L, x)                  // insert into max-heap
   else:
      push(U, x)                  // insert into min-heap
   // Rebalance
   if size(L) > size(U) + 1:
      push(U, extractMax(L))
   elif size(U) > size(L):
      push(L, extractMin(U))

function Delete(x):
   idx = positionMap[x]           // O(1) lookup via hash map
   if x <= peek(L):               // x is in lower heap
      removeFromHeap(L, idx)      // swap-and-sift → O(log n)
   else:                          // x is in upper heap
      removeFromHeap(U, idx)      // swap-and-sift → O(log n)
   // Rebalance
   if size(L) > size(U) + 1:
      push(U, extractMax(L))
   elif size(U) > size(L):
      push(L, extractMin(U))
```

5.4 Complexity Analysis

Operation	Time Complexity	Notes
Insert(x)	O(log n)	Heap push + at most one rebalance move
Delete(x)	O(log n)	Requires position map for O(1) lookup; then sift up/down
FindMedian()	O(1)	Root access only — no traversal needed

Space complexity: O(n) — all n elements are stored across the two heaps, plus O(n) for the position map.

Question 6: Red-Black Tree Height Bound

6.1 Definition: Black-Height

The black-height of a node x, written bh(x), is the number of BLACK nodes on any simple path from x (exclusive, i.e. not counting x itself) down to any descendant NIL leaf.

This is well-defined because RBT Property 5 guarantees that every root-to-leaf path has the same count of black nodes — so the choice of path does not affect the result. The black-height of the tree is bh(root).

Example: If a subtree root has bh = 3, then every path from that node down to any NIL leaf passes through exactly 3 black nodes (not counting the root itself).

6.2 Lemma: A Subtree with Black-Height bh Has ≥ 2^bh − 1 Internal Nodes

Proof by strong induction on the height of the subtree:

Base case (height 0):

The subtree is a single NIL leaf. It has black-height bh = 0 and zero internal nodes.

$0 = 2^0 - 1 = 0$. ✓

Inductive step:

Assume the lemma holds for all subtrees of height < h. Consider a subtree rooted at node x with bh(x) = bh.

- Node x has two children, each the root of a subtree of height < h.
- If a child is RED, its black-height equals bh(x): bh(child) = bh (it contributes nothing to the count).
- If a child is BLACK, it contributes one to the count: bh(child) = bh − 1.
- In both cases: bh(child) ≥ bh − 1.

By the induction hypothesis, each child subtree has at least 2^(bh−1) − 1 internal nodes. Counting all nodes in the subtree rooted at x:

```
nodes(x) ≥ 1  (node x itself)
       + (2^(bh−1) − 1)  (left subtree, by IH)
       + (2^(bh−1) − 1)  (right subtree, by IH)
     = 1 + 2·(2^(bh−1) − 1)
     = 1 + 2^bh − 2
     = 2^bh − 1  ✓
```

Conclusion: A subtree with black-height bh contains at least 2^bh − 1 internal nodes. □

6.3 Proof: $h \leq 2 \cdot \log_2(n + 1)$

Let h be the height of the Red-Black Tree and bh the black-height of the root.

Step 1 — Bound black-height from height:

Property 4 prohibits two consecutive red nodes. On any root-to-leaf path of length h, at most half the nodes can be red, so at least half must be black:

$$bh \geq h / 2$$

Step 2 — Bound n from black-height:

From the Lemma in 6.2, the entire tree with black-height bh satisfies:

$$n \geq 2^{bh} - 1 \quad n + 1 \geq 2^{bh}$$

Step 3 — Combine the two bounds:

```
n + 1 ≥ 2^bh ≥ 2^(h/2)      (substituting bh ≥ h/2)

Take log₂ of both sides:
  log₂(n + 1) ≥ h / 2
  h ≤ 2 · log₂(n + 1)  ✓
```

Proven: The height of any Red-Black Tree with n nodes satisfies $h \leq 2 \cdot \log_2(n + 1)$. □

6.4 Why This Guarantees O(log n) Search

Binary tree search visits at most one node per level, so search time is proportional to tree height:

$$\text{Search time} \leq c \cdot h \leq c \cdot 2 \cdot \log_2(n + 1) = O(\log n)$$

- No degeneration: the bound holds for every possible insertion sequence, not just random inputs.
- Constant factor: the factor of 2 is absorbed by O-notation — the asymptotic class is identical to a perfectly balanced BST.
- **Practical implication:** for $n = 10^6$ nodes, the maximum path length is $\leq 2 \times 20 = 40$ comparisons, regardless of insertion order.

Question 7: Adversarial Input

7.1 Hash Tables with Linear Probing

Adversarial sequence (table size m = 10): 10, 20, 30, 40, 50, 60, 70, 80, 90, 100

General form: m, 2m, 3m, …, nm

Why this forces worst-case:

Every key satisfies $h(k) = k \bmod m = 0$, mapping all keys to bucket 0. Linear probing resolves collisions by scanning sequentially: 0, 1, 2, 3, …

- The 1st insertion probes 1 slot (bucket 0 empty → insert).
- The 2nd insertion probes 2 slots (0 taken → probe 1).
- The i-th insertion probes i slots.
- Total probes = $1 + 2 + 3 + \ldots + n = n(n+1)/2 = \Theta(n^2)$.
- Searching for a non-existent key must scan the entire primary cluster → O(n) per search.

Linear probing trace — table size 10, inserting 10, 20, 30, 40:

```
Insert 10: h(10)=0  → slot 0 empty  → place at 0  (1 probe)
Insert 20: h(20)=0  → slot 0 taken  → probe 1 (empty)  → place at 1  (2 probes)
Insert 30: h(30)=0  → slots 0,1 taken  → probe 2 (empty)  → place at 2  (3 probes)
Insert 40: h(40)=0  → slots 0,1,2 taken  → probe 3 (empty)  → place at 3  (4 probes)

Table: [10][20][30][40][ ][ ][ ][ ][ ][ ]
Index:   0  1  2  3  4  5  6  7  8  9

Primary cluster at bucket 0 grows with each insertion.
Total probes for 4 inserts = 1+2+3+4 = 10 = O(n²) for n inserts.
```

Figure 5 — Primary clustering forces $O(n^2)$ total probes

7.2 Quicksort — Adversarial Input

Assumption

The following analysis applies to naïve quicksort where the pivot is always the last element. Sorted input is NOT worst-case for randomised-pivot or median-of-three variants — addressed at the end of this section.

Adversarial sequence: 1, 2, 3, 4, 5, …, n (sorted ascending)

Reverse-sorted input n, n−1, …, 2, 1 is equally adversarial for the same reason.

Why sorted input forces worst-case:

- Pivot = last element = largest element in the subarray.
- Partitioning produces subproblems of size 0 (left — nothing is smaller than the maximum) and n−1 (right — everything else).
- No balanced split ever occurs — the recursion depth equals n.
- Recurrence: $T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n) \rightarrow T(n) = \Theta(n^2)$.

Quicksort partition trace on [1, 2, 3, 4, 5] (pivot = last element):

```
[1, 2, 3, 4, 5]   pivot=5  → left=[1,2,3,4], right=[]   (depth 1)
[1, 2, 3, 4]      pivot=4  → left=[1,2,3],   right=[]   (depth 2)
[1, 2, 3]         pivot=3  → left=[1,2],     right=[]   (depth 3)
[1, 2]            pivot=2  → left=[1],       right=[]   (depth 4)
[1]               base case                             (depth 5)

Recursion depth = n = 5.  Every level does Θ(n) work.
Total comparisons = (n-1)+(n-2)+…+1 = n(n-1)/2 = Θ(n²).
```

Figure 6 — Sorted input produces maximally unbalanced partitions

Other pivot strategies:

- Random pivot: no deterministic adversarial input exists — the adversary cannot predict the random choice in advance. Expected O(n log n) on all inputs.
- Median-of-three pivot: sorted and reverse-sorted inputs are no longer adversarial. A specific organ-pipe pattern (e.g. 1, 3, 2, 5, 4, 7, 6, …) can defeat median-of-three selection and still force $O(n^2)$.

Question 8: Programming Assignment

The full implementation for this programming assignment — including all source code, experimental setup, dataset generation, performance results, and discussion — is available in the following GitHub repository: https://github.com/limhpone/125970-Assignment-3-ADA.git

The repository README provides a complete account of all submission requirements, covering the following:

- Implementation approach — design decisions for the BST, Red-Black Tree, and Hash Table, including the iterative BST, CLRS-style RBT with NIL sentinel, and separate-chaining hash table.
- Experimental setup — language, libraries, machine specifications, and methodology for timing insert and search operations.
- Dataset design — how the three input patterns (random, sorted, and adversarial) were generated for dataset sizes of 10^3, 10^4, and 10^5.
- Performance results — measured insertion and search times presented in tables and plots, alongside observed tree heights and hash table collision statistics.
- Discussion and interpretation — analysis of whether empirical results match theoretical time complexity predictions, including notes on any discrepancies.